# 计算机系统 I：RISC-V ISA 与汇编语言
# 复习讲义

## 目录

**1 RISC-V ISA 概览 (RISC-V ISA Overview) 2**
1.1 什么是 RISC-V? . . . 2
1.2 RISC-V 的设计目标与原则 . . . 2
1.3 用户级 ISA 与标准扩展 . . . 3
1.4 基础 RISC-V 与处理器状态 . . . 3
1.5 RV32I 指令速览 (RV32I Instruction Quick Reference) . . . 4
1.6 动态指令混合 (Dynamic Instruction Mix) . . . 5

**2 指令编码与格式 (Instruction Encoding and Formats) 6**
2.1 混合长度编码 . . . 6
2.2 核心字段位置 . . . 6
2.3 六类常用指令格式 . . . 7
2.4 立即数编码变体 . . . 7
2.5 RISC-V 寻址方式 . . . 7

**3 RV32I 指令系统 (RV32I Instruction Set) 8**
3.1 RV32I 指令类别总览 . . . 8
3.2 ALU 指令：R-Type . . . 8
3.3 按位逻辑与移位 . . . 9
3.4 ALU 指令：I-Type . . . 9
3.5 ALU 指令：U-Type 与大立即数 . . . 9

**4 内存访问、字节序与对齐 (Memory Access, Endianness and Alignment) 10**
4.1 Load/Store 架构 . . . 10
4.2 常用 Load/Store 指令 . . . 10
4.3 字节序 (Endianness) . . . 11
4.4 内存对齐 (Memory Alignment) . . . 11
4.5 数组寻址例：`g = h + A[i]` . . . 11

**5 控制转移指令 (Control Transfer Instructions) 12**
5.1 跳转与分支的共同特点 . . . 12
5.2 `jal` 与 `jalr` . . . 12
5.3 条件分支 B-Type . . . 12
5.4 编译 if 语句 . . . 13
5.5 更多条件与边界检查技巧 . . . 13
5.6 编译 while 循环 . . . 14
5.7 远距离分支 . . . 14
5.8 switch 与跳转表 . . . 14
5.9 NOP 与机器码解码 . . . 15
5.10 立即数编码变体 (Immediate Encoding Variants) . . . 15

**6 指令编码速查 (Encoding Reference) 17**
6.1 常用 R-Type 与 I-Type 编码 . . . 17
6.2 Load/Store、Branch、U/J-Type 编码 . . . 17

**7 特权 ISA 与软件栈 (Privileged ISA and Software Stack) 18**
7.1 特权模式 . . . 18
7.2 实现层次与特权软件栈 . . . 18

**8 汇编语言与调用约定 (Assembly Language and Calling Convention) 18**
8.1 函数调用角色 . . . 18
8.2 函数调用六步 . . . 19
8.3 函数入口与出口 . . . 19
8.4 寄存器约定 . . . 19
8.5 栈与栈指针 . . . 20
8.6 叶函数示例 . . . 20
8.7 非叶函数与递归 . . . 21
8.8 递归的缺点 . . . 22
8.9 调用前后保持与不保持的内容 . . . 22
8.10 C 变量存储类别与内存布局 . . . 22
8.11 栈帧 (Stack Frame) . . . 22

**9 从源代码到运行程序 (From Source Code to A Running Program) 23**
9.1 整体流程 . . . 23
9.2 编译器与预处理器 . . . 23
9.3 汇编器与伪指令 . . . 23
9.4 汇编指示符 (Assembler Directives) . . . 24
9.5 Hello World 汇编结构 . . . 24

**10 ELF、链接与装载 (ELF, Linking and Loading) 25**
10.1 ELF 基础 . . . 25
10.2 三类目标文件 . . . 25
10.3 可重定位目标文件 . . . 25
10.4 链接器的作用 . . . 26
10.5 地址修正与符号解析 . . . 26
10.6 可执行文件格式 . . . 27
10.7 静态链接与动态链接 . . . 27
10.8 静态库与共享库 . . . 27
10.9 装载器 (Loader) . . . 27
10.10动态装载、PIC 与 GOT . . . 28
10.11Lazy Linkage . . . 28

**11 补充主题与课件结尾 (Additional Topics) 28**
11.1 编程语言演进与自举 . . . 28
11.2 Reflections on Trusting Trust . . . 29
11.3 真实入口点：`_start` 与 `crt0` . . . 29
11.4 Quiz：符号放在哪里? . . . 29

**12 全章总结 (Summary) 30**

# 1 RISC-V ISA 概览 (RISC-V ISA Overview)

## 1.1 什么是 RISC-V?

RISC-V 读作 “risk-five”，是一套开放的精简指令集计算机架构标准 (ISA, Instruction Set Architecture)。这里的 ISA 是软硬件之间的接口：软件看到的是寄存器、指令、异常和内存访问规则；硬件可以用不同的微结构去实现同一套规则。

- **RISC-V 是 ISA 标准**：它规定指令如何编码、寄存器如何命名、程序如何跳转和访问内存，而不是某一个固定芯片。
- **RISC-V 是开放 ISA**：任何个人、学校或公司都可以实现兼容的处理器和软件生态。
- **与常见商业 ISA 对比**：x86、ARM、Power、MIPS、SPARC 等通常受商业授权、专利或生态控制约束；RISC-V 设计目标之一是减少这些限制。
- **起源**：2010 年由 UC Berkeley 的 Krste Asanović、David Patterson 及学生发起。
- **规范版本**：课件提到非特权规范 Unprivileged Spec 20191213 (v2.2)，特权规范 Privileged Spec 20211203 (v1.11)，更多规范见 https://riscv.org/ 与 https://github.com/riscv/riscv-isa-manual。

**ISA 与实现的关系**

ISA 关心 “程序员可见” 的抽象，例如 `add x9,x20,x21` 的语义；微结构关心 “如何实现”，例如流水线、乱序执行、缓存和分支预测。RISC-V 的目标是避免为某一种微结构或某一种工艺过度定制。

## 1.2 RISC-V 的设计目标与原则

RISC-V 在定义时追求以下目标：

- **完全开放**：对学术界和工业界免费开放。
- **真实可实现**：不是只用于模拟或二进制翻译，而是适合直接硬件实现。
- **不过度架构化**：不绑定微码、顺序、解耦、乱序等某一种实现风格，也不绑定 full-custom、ASIC、FPGA 等某一种技术。
- **小而可扩展**：基础整数 ISA 很小，可单独用于教学、研究和定制加速器；通用软件再按需加入标准扩展。
- **支持 IEEE 754-2008 浮点标准**：浮点扩展与现代科学计算需求配合。

RISC-V 的命名方式通常为：

$$\text{RV} + \text{字长} + \text{扩展集合}$$

例如 `RV32I` 表示 32 位 RISC-V，支持基础整数指令集 `I`；`RV64G` 表示 64 位 RISC-V，支持通用集合 `G`。

### 1.3 用户级 ISA 与标准扩展

用户级 ISA 定义普通计算所需的指令：

- **基础整数 ISA** `I`：ALU 运算、分支/跳转、Load/Store。
- `M`：整数乘法与除法。
- `A`：原子指令，用于并发同步。
- `F`：单精度浮点。
- `D`：双精度浮点。
- `C`：压缩指令，16 位长度。
- `G = IMAFD`：基础整数指令加四个常用标准扩展。
- **自定义扩展**：用于领域专用加速器或研究。

### 1.4 基础 RISC-V 与处理器状态

RISC-V 有 32 位和 64 位地址空间变体：

- **RV32**：32 位地址空间，寄存器宽度通常为 32 位。
- **RV64**：64 位地址空间，寄存器宽度通常为 64 位。
- 常见组合包括 `RV32IM`、`RV32IMA`、`RV32IMAFD`、`RV32G` 等。

RISC-V 处理器状态包括：

- **程序计数器 PC**：保存当前/下一条指令地址。
- **32 个整数寄存器** `x0`–`x31`：
  - `x0` 永远为 0，写入会被丢弃。
  - `x1` 常作为返回地址寄存器 `ra`。
- **32 个浮点寄存器** `f0`–`f31`：可保存单精度或双精度 IEEE 浮点数。
- **浮点状态寄存器** `fsr`：记录浮点舍入模式与异常信息。

## 1.5 RV32I 指令速览 (RV32I Instruction Quick Reference)

RV32I 是 32 位 RISC-V 的基础整数指令集，课件用四页表格先给出整体轮廓：整数计算、Load/Store、控制转移与少量杂项系统指令。这里先做一个总览，后续章节再展开编码、寻址和调用约定。

### RV32I 基础类别

| 类别 | 典型指令族与含义 |
|---|---|
| Integer Computation | `add/addi`，`sub`；`and/andi`、`or/ori`、`xor/xori`；`sll/slli`、`sra/srai`、`srl/srli`；`lui`，`auipc`；`slt/slti`、`sltu/sltiu`。 |
| Loads and Stores | `lb/lh/lw` 与 `sb/sh/sw` 访问 byte、halfword、word；`lbu/lhu` 对 byte、halfword 做零扩展读取。RV64 还会加入 `ld/sd/lwu` 等 doubleword 相关指令。 |
| Control Transfer | `beq/bne` 比较相等或不等；`blt/bge` 做有符号小于/大于等于判断；`bltu/bgeu` 做无符号比较；`jal` 与 `jalr` 完成跳转和函数调用/返回。 |
| Miscellaneous | `fence` 约束 load/store 的内存可见顺序；`fence.i` 同步指令流与数据写入；`ecall/ebreak` 进入执行环境或调试断点；CSR 指令读写、置位或清位控制状态寄存器。 |

**RV32I 的复习边界**

RV32I 的核心是“寄存器计算 + 显式内存访问 + PC 相对控制流”。乘除法、原子操作和浮点运算不属于 RV32I 本体，而是分别放在 `M/A/F/D` 等扩展中。

### ALU 指令示例

基础 ALU 指令既可使用寄存器-寄存器操作数，也可使用一个立即数操作数。课件表格中的典型例子如下：

| Example instruction | Instruction name | Meaning |
|---|---|---|
| `add x1,x2,x3` | Add | $x1 \leftarrow x2 + x3$ |
| `addi x1,x2,3` | Add immediate | $x1 \leftarrow x2 + 3$ |
| `lui x1,42` | Load upper immediate | 把 20 位立即数放入高位，低 12 位补 0；在 RV64 中结果还会按第 31 位向高位符号扩展。 |
| `sll x1,x2,5` | Shift left logical | $x1 \leftarrow x2 \ll 5$ |
| `slt x1,x2,x3` | Set less than | 若 $x2 < x3$，则 $x1 = 1$，否则 $x1 = 0$。 |

补充：严格的基础 RISC-V 中，常数移位使用 I-Type 的 `slli/srli/srai`；`sll rd,rs1,rs2` 的移位量来自寄存器 `rs2` 的低位。许多教材表格会用 `sll x1,x2,5` 表达“左移 5 位”的语义，实际汇编时通常写作 `slli x1,x2,5`。

**Load/Store 指令示例**

所有内存引用都使用基址加位移形式 `imm(rs1)`。短于寄存器宽度的 Load 分为符号扩展和零扩展两类；Store 只把寄存器低位写回内存。

| Example instruction | Instruction name | Meaning |
|---|---|---|
| `ld x1,80(x2)` | Load doubleword | `x1` ← Mem[80 + `x2`]。RV64 指令。 |
| `lw x1,60(x2)` | Load word | 读取 32 位 word，并符号扩展到寄存器宽度。 |
| `lwu x1,60(x2)` | Load word unsigned | 读取 32 位 word，并零扩展到 64 位。RV64 指令。 |
| `lb x1,40(x3)` | Load byte | 读取 8 位 byte，并符号扩展。 |
| `lbu x1,40(x3)` | Load byte unsigned | 读取 8 位 byte，并零扩展。 |
| `lh x1,40(x3)` | Load halfword | 读取 16 位 halfword，并符号扩展。 |
| `flw f0,50(x3)` | Load FP single | 读取单精度浮点到浮点寄存器。 |
| `fld f0,50(x2)` | Load FP double | 读取双精度浮点到浮点寄存器。 |
| `sd x2,400(x3)` | Store doubleword | 把 `x2` 的 64 位写入内存。RV64 指令。 |
| `sw x3,500(x4)` | Store word | 把 `x3` 的低 32 位写入内存。 |
| `fsw f0,40(x3)` | Store FP single | 把浮点寄存器中的单精度值写入内存。 |
| `fsd f0,40(x3)` | Store FP double | 把浮点寄存器中的双精度值写入内存。 |
| `sh x3,502(x2)` | Store halfword | 把 `x3` 的低 16 位写入内存。 |
| `sb x2,41(x3)` | Store byte | 把 `x2` 的低 8 位写入内存。 |

注意：RV32I 本体只包含整数寄存器和 32 位地址空间中的基础 Load/Store；上表中的 `ld/sd/lwu` 属于 RV64，`flw/fld/fsw/fsd` 属于浮点扩展。课件把它们放在同一张表中，是为了展示 RISC-V Load/Store 家族的统一寻址风格。

**控制转移指令示例**

| Example instruction | Instruction name | Meaning |
|---|---|---|
| `jal x1,offset` | Jump and link | `x1` ← `PC` + 4，`PC` ← `PC` + (`offset` ≪ 1)。 |
| `jalr x1,x2,offset` | Jump and link register | `x1` ← `PC` + 4，`PC` ← `x2` + `offset`。 |
| `beq x3,x4,offset` | Branch equal | 若 `x3` = `x4`，则 `PC` ← `PC`+(`offset` ≪ 1)。 |
| `bgt x3,x4,name` | Branch greater than | 若 `x3` > `x4`，则跳转到 `name`。这是常见伪指令，通常可由 `blt x4,x3,name` 展开。 |

除了 `jalr` 这种寄存器间接跳转外，RISC-V 控制流指令通常都是 PC-relative，因此目标位置随程序整体搬移而自然改变，便于生成位置无关代码。

## 1.6 动态指令混合 (Dynamic Instruction Mix)

SPECint2006 中 RISC-V 动态指令大致呈现如下分布。该表的意义是帮助理解为什么 Load/Store、分支和 ALU 是基础 ISA 的核心。

| Program | Loads | Stores | Branches | Jumps | ALU ops |
|---|---|---|---|---|---|
| astar | 28% | 6% | 18% | 2% | 46% |
| bzip | 20% | 7% | 11% | 1% | 54% |
| gcc | 17% | 23% | 20% | 4% | 36% |
| gobmk | 21% | 12% | 14% | 2% | 50% |
| h264ref | 33% | 14% | 5% | 2% | 45% |
| hmmer | 28% | 9% | 17% | 0% | 46% |
| libquantum | 16% | 6% | 29% | 0% | 48% |
| mcf | 35% | 11% | 24% | 1% | 29% |
| omnetpp | 23% | 15% | 17% | 7% | 31% |
| perlbench | 25% | 14% | 15% | 7% | 39% |
| sjeng | 19% | 7% | 15% | 3% | 56% |
| xalancbmk | 30% | 8% | 27% | 3% | 31% |

典型整数程序中，Load 频率常为 Store 频率的 1/5 到 3 倍；`gcc` 中 Store 比 Load 更高，是课件指出的一个异常现象。

# 2 指令编码与格式 (Instruction Encoding and Formats)

## 2.1 混合长度编码

RISC-V 支持 16、32、48、64 位以及更长的混合长度指令编码。基础 RV32 指令固定为 32 位，并且最低两位为 $11_2$。压缩扩展 `C` 使用 16 位指令，因此分支和跳转目标统一按 16 位粒度对齐，即使在只使用 32 位基础指令时也如此。

- 16 位指令：低 2 位不为 $11_2$。
- 32 位指令：低 2 位为 $11_2$，但低 3 位不全为 1。
- 48/64 位及更长指令：用特定低位模式继续扩展。

## 2.2 核心字段位置

RISC-V 指令编码的一个关键设计是：寄存器字段位置尽量固定，立即数符号位总在指令第 31 位。

- **`opcode`**：低 7 位，决定指令大类；基础 32 位指令中低 2 位为 $11_2$。
- **`rd`**：目的寄存器。
- **`rs1`, `rs2`**：源寄存器。
- **`funct3`, `funct7`**：在同一 opcode 大类下进一步区分具体运算。
- **立即数字段**：为保持寄存器字段固定，S/B/J 类型的立即数会被拆开重排。

## 2.3 六类常用指令格式

| 格式 | 31..25 | 24..20 | 19..15 | 14..12 | 11..7 | 6..0 | 用途 |
|---|---|---|---|---|---|---|---|
| R-type | funct7 | rs2 | rs1 | funct3 | rd | opcode | 寄存器-寄存器算术/逻辑 |
| I-type | imm[11:0] | | rs1 | funct3 | rd | opcode | Load、立即数算术、jalr |
| S-type | imm[11:5] | rs2 | rs1 | funct3 | imm[4:0] | opcode | Store |
| B-type | imm[12,10:5] | rs2 | rs1 | funct3 | imm[4:1,11] | opcode | 条件分支 |
| U-type | imm[31:12] | | | | rd | opcode | 高 20 位立即数 |
| J-type | imm[20,10:1,11,19:12] | | | | rd | opcode | 无条件跳转 jal |

**编码复习口诀**

寄存器字段 rd/rs1/rs2 的位置尽量不变；立即数为适配硬件与对齐规则被拆散；所有立即数的符号位放在 inst[31]，便于统一符号扩展。

## 2.4 立即数编码变体

I、S、B、U、J 格式都含立即数，但使用方式不同：

- **I-immediate**：$\{\texttt{inst[31]}\ 扩展, \texttt{inst[30:20]}\}$，常用于 addi、Load、jalr。
- **S-immediate**：$\{\texttt{inst[31]}\ 扩展, \texttt{inst[30:25]}, \texttt{inst[11:7]}\}$，用于 Store。
- **B-immediate**：$\{\texttt{inst[31]}\ 扩展, \texttt{inst[7]}, \texttt{inst[30:25]}, \texttt{inst[11:8]}, 0\}$，最低位恒为 0，表示按 2 字节对齐的分支偏移。
- **U-immediate**：$\{\texttt{inst[31:12]}, 12'b0\}$，用于构造高位立即数。
- **J-immediate**：$\{\texttt{inst[31]}\ 扩展, \texttt{inst[19:12]}, \texttt{inst[20]}, \texttt{inst[30:21]}, 0\}$，用于 jal 的 PC 相对跳转。

## 2.5 RISC-V 寻址方式

课件总结了四类常见寻址方式：

1. **立即数寻址 (Immediate Addressing)**：操作数直接来自指令中的立即数字段，例如 addi x1,x2,3。
2. **寄存器寻址 (Register Addressing)**：操作数来自寄存器，例如 add x1,x2,x3。
3. **基址/位移寻址 (Base/Displacement Addressing)**：有效地址为 rs1 + imm，Load/Store 使用这一种内存寻址方式。
4. **PC 相对寻址 (PC-relative Addressing)**：目标地址为 PC + offset，用于分支、jal、auipc 等。

# 3 RV32I 指令系统 (RV32I Instruction Set)

## 3.1 RV32I 指令类别总览

RV32I 的基础指令可以按功能分为：

- **整数计算**：`add/addi`、`sub`、`and/andi`、`or/ori`、`xor/xori`、`sll/slli`、`srl/srli`、`sra/srai`、`slt/slti`、`sltu/sltiu`、`lui`、`auipc`。
- **控制转移**：`beq`、`bne`、`blt`、`bge`、`bltu`、`bgeu`、`jal`、`jalr`。
- **Load/Store**：`lb/lh/lw/lbu/lhu` 与 `sb/sh/sw`；RV64 还包括 `ld/sd/lwu`。
- **杂项与系统**：`fence`、`fence.i`、`ecall`、`ebreak`、CSR 读写/置位/清位及其立即数形式。

## 3.2 ALU 指令：R-Type

R-Type 指令使用两个源寄存器和一个目的寄存器：

$$\texttt{rd} \leftarrow \texttt{rs1}\ \text{op}\ \texttt{rs2}$$

典型指令包括：

- **`add/sub`**：整数加法与减法。
- **`slt/sltu`**：若 `rs1` $<$ `rs2` 则 `rd=1`，否则 `rd=0`；`slt` 按有符号比较，`sltu` 按无符号比较。
- **`sll/srl/sra`**：逻辑左移、逻辑右移、算术右移。
- **`and/or/xor`**：按位与、或、异或。

**机器码例子：`add x9,x20,x21`**

RISC-V 汇编：

```
add x9, x20, x21
```

字段十进制形式：

$$|\ 0\ |\ 21\ |\ 20\ |\ 0\ |\ 9\ |\ 51\ |$$

字段二进制形式：

$$|\ 0000000\ |\ 10101\ |\ 10100\ |\ 000\ |\ 01001\ |\ 0110011\ |$$

完整机器码：

$$0000\ 0001\ 0101\ 1010\ 0000\ 0100\ 1011\ 0011_2 = \texttt{0x015A04B3}$$

## 3.3 按位逻辑与移位

**and：掩码操作**

and 常用于选择某些位并把其他位清零：

```
and x9, x10, x11
```

只有两个操作数同一位都为 1 时，结果对应位才为 1。

**xor：差异与取反**

xor 可用于翻转部分位。当掩码全 1 时，异或相当于按位取反：

```
xor x9, x10, x12     # if x12 = -1, x9 = ~x10
```

**移位运算**

- **逻辑左移 sll/slli**：低位补 0。左移 $i$ 位相当于乘以 $2^i$（不考虑溢出）。
- **逻辑右移 srl/srli**：高位补 0，适合无符号数。
- **算术右移 sra/srai**：高位补符号位，适合有符号数。

例：$9_{10} = 0000\cdots1001_2$，左移 4 位得到 $0000\cdots1001\,0000_2 = 144_{10}$：

```
slli x11, x19, 4     # x11 = x19 << 4
```

## 3.4 ALU 指令：I-Type

I-Type 使用一个源寄存器和一个 12 位立即数。所有 RISC-V 指令中的立即数都会按规则进行符号扩展。

- **addi**：rd = rs1 + sign-extended imm。
- **slti/sltiu**：与立即数比较并设置 0/1。
- **andi/ori/xori**：逻辑立即数运算。
- **slli/srli/srai**：按常数位数移位，移位量来自立即数字段的低位。

## 3.5 ALU 指令：U-Type 与大立即数

lui 与 auipc 使用 U-Type，用 20 位立即数构造高位：

- **lui rd,imm20**：把 20 位立即数写入目的寄存器高 20 位，低 12 位清 0。RV64 中还会把第 31 位符号扩展到高 32 位。
- **auipc rd,imm20**：rd = PC + (imm20 << 12)，常用于 PC 相对地址生成。

由于 I-Type 立即数只有 12 位且会符号扩展，加载 32 位常量常用两条指令完成：

```
lui  s3, 976
addi s3, s3, 2304
```

其中 `lui` 生成高 20 位，`addi` 加入低 12 位。若低 12 位最高位为 1，要注意 `addi` 的符号扩展可能影响高位，实际汇编器会在伪指令 `li` 中处理这些细节。

# 4 内存访问、字节序与对齐 (Memory Access, Endianness and Alignment)

## 4.1 Load/Store 架构

RISC-V 是典型 Load/Store 架构：算术逻辑运算只能在寄存器之间进行，内存访问由专门的 Load/Store 指令完成。

$$\text{Load:} \quad \texttt{rd} \leftarrow \text{MEM}[\texttt{rs1} + \texttt{imm}]$$

$$\text{Store:} \quad \text{MEM}[\texttt{rs1} + \texttt{imm}] \leftarrow \texttt{rs2}$$

- Load 使用 I-Type。
- Store 使用 S-Type，立即数被拆成 `imm[11:5]` 与 `imm[4:0]` 两段。
- RISC-V 使用字节地址，连续 doubleword 地址相差 8。

## 4.2 常用 Load/Store 指令

| 类别 | 指令 | 含义 |
|---|---|---|
| Load | `lb/lh/lw` | 读取 byte/halfword/word，并按符号扩展到寄存器宽度。 |
| Load unsigned | `lbu/lhu` | 读取 byte/halfword，并用 0 扩展。RV64 还有 `lwu`。 |
| RV64 Load | `ld` | 读取 doubleword，64 位。 |
| Store | `sb/sh/sw` | 将寄存器低 8/16/32 位写入内存。 |
| RV64 Store | `sd` | 将寄存器 64 位写入内存。 |
| 浮点 Load/Store | `flw/fld/fsw/fsd` | 读取或写入单精度/双精度浮点寄存器。 |

例子：

```
ld x9, 64(x22)      # x9 = MEM[x22 + 64]
sd x9, 64(x22)      # MEM[x22 + 64] = x9
```

### 4.3 字节序 (Endianness)

当一个多字节数据放入内存时，字节序决定高低字节与地址大小的对应关系。

- **大端序 (Big Endian)**：高字节放在低地址，低字节放在高地址。PowerPC 是课件示例之一。
- **小端序 (Little Endian)**：低字节放在低地址，高字节放在高地址。RISC-V 通常采用小端序。

以 32 位数 `0x12345678` 从地址 `0x100` 开始存放为例：

| 字节序 | 0x100 | 0x101 | 0x102 | 0x103 |
|---|---|---|---|---|
| Big Endian | 12 | 34 | 56 | 78 |
| Little Endian | 78 | 56 | 34 | 12 |

### 4.4 内存对齐 (Memory Alignment)

对齐约束要求某些数据必须放在特定边界上，例如 4 字节整数通常放在 4 字节边界，8 字节 doubleword 通常放在 8 字节边界。

- 对齐访问通常更快，硬件一次内存访问即可读出完整数据。
- 未对齐访问可能需要多次内存访问，甚至在某些实现上触发异常。
- 编译器会在结构体成员之间插入填充字节 (padding)，以满足对齐要求。

课件中的结构体例子：

```
struct {
    int a;
    char b;
    char c[2];
    char d[3];
    float e;
};
```

若不考虑对齐，`float e` 可能跨越自然边界，无法一次读出；加入 padding 后可保证 `e` 对齐，但会牺牲部分空间。复习时要能判断“哪些空洞是为了对齐而插入的”。

### 4.5 数组寻址例：`g = h + A[i]`

假设 `A` 是包含 100 个 doubleword 的数组，`g,h,i` 分别在 `x1,x2,x4`，数组基地址在 `x3`。由于每个 doubleword 为 8 字节，需要先计算 $8i$：

```
add x5, x4, x4     # x5 = 2 * i
add x5, x5, x5     # x5 = 4 * i
add x5, x5, x5     # x5 = 8 * i
add x5, x5, x3     # x5 = &A[i]
ld  x6, 0(x5)      # x6 = A[i]
add x1, x2, x6     # g = h + A[i]
```

补充：也可用 `slli x5,x4,3` 更直接地计算 $8i$。

# 5 控制转移指令 (Control Transfer Instructions)

## 5.1 跳转与分支的共同特点

RISC-V 控制转移指令没有架构可见的延迟槽 (delay slot)。除 `jalr` 使用寄存器间接跳转外，分支和 `jal` 通常使用 PC 相对寻址：

$$\text{target} = \texttt{PC} + \text{offset}$$

由于目标按 16 位粒度对齐，B-Type 和 J-Type 立即数最低位隐含为 0。

## 5.2 `jal` 与 `jalr`

- **`jal rd,label`**：保存返回地址 `PC`$+4$ 到 `rd`，然后跳转到 PC 相对目标。函数调用常用 `jal x1,ProcedureLabel`。
- **`jalr rd,imm(rs1)`**：保存返回地址 `PC`$+4$ 到 `rd`，目标地址为 `rs1`$+$`imm`。返回常用 `jalr x0,0(x1)`。
- **丢弃返回地址**：若 `rd=x0`，返回地址被丢弃，可用于无条件跳转。

常见伪指令：

```
j label       # jal x0, label
ret           # jalr x0, 0(x1)
```

## 5.3 条件分支 B-Type

条件分支比较两个寄存器，根据条件决定是否跳转：

| 指令 | 含义 |
|---|---|
| `beq rs1,rs2,L` | 相等则跳转 |
| `bne rs1,rs2,L` | 不等则跳转 |
| `blt rs1,rs2,L` | 有符号 $rs1 < rs2$ 则跳转 |
| `bge rs1,rs2,L` | 有符号 $rs1 \geq rs2$ 则跳转 |
| `bltu rs1,rs2,L` | 无符号 $rs1 < rs2$ 则跳转 |
| `bgeu rs1,rs2,L` | 无符号 $rs1 \geq rs2$ 则跳转 |

课件例子：`bne x10,x11,2000` 中，偏移字段编码的是以半字为单位的 PC 相对偏移。若目标地址差为 12 字节，编码字段应表示 $12/2 = 6$，不是 12。

### 5.4 编译 if 语句

C 代码：

```
if (i == j)
    f = g + h;
else
    f = g - h;
```

假设 `f` 到 `j` 分别在 `x19` 到 `x23`：

```
bne x22, x23, ELSE
add x19, x20, x21
beq x0, x0, EXIT
ELSE:
sub x19, x20, x21
EXIT:
```

其中 `beq x0,x0,EXIT` 是永真分支，也可写成伪指令 `j EXIT`。

### 5.5 更多条件与边界检查技巧

- `if (a > b) a += 1;` 可改写为先测试退出条件：

```
bge  x23, x22, Exit     # if b >= a, skip
addi x22, x22, 1
Exit:
```

边界检查快捷写法：

```
bgeu x20, x11, IndexOutofBounds
```

它能同时捕捉 `x20` $\geq$ `x11` 和 `x20` $< 0$。原因是负数按无符号数解释会变成很大的数。

### 5.6 编译 while 循环

C 代码:

```
while (save[i] == k)
    i = i + 1;
```

假设 i,k 在 x22,x24，数组 save 基地址在 x25:

```
Loop:
    slli x10, x22, 3
    add  x10, x10, x25
    ld   x9, 0(x10)
    bne  x9, x24, Exit
    addi x22, x22, 1
    beq  x0, x0, Loop
Exit:
```

### 5.7 远距离分支

若条件分支目标太远，超出 B-Type 可编码范围，可以反转条件，让短分支跳过一个长跳转:

```
# original
beq x10, x0, L1

# rewritten
bne x10, x0, L2
jal x0, L1
L2:
```

### 5.8 switch 与跳转表

对于密集的 case，编译器可使用跳转地址表。假设 k 在 x25，x5 中有 4，跳转表基地址在 x6:

```
blt  x25, x0, Exit
bge  x25, x5, Exit
slli x7, x25, 3
add  x7, x7, x6
ld   x7, 0(x7)
jalr x1, 0(x7)
Exit:
```

每个 case 代码段执行完后可用 `jalr x0,0(x1)` 返回到 switch 之后。这里 `jalr` 也体现了它可用于 computed jump。

## 5.9 NOP 与机器码解码

RISC-V 的 NOP 通常是：

```
addi x0, x0, 0
```

它不改变任何可见状态，因为 `x0` 恒为 0 且写入无效。

机器码解码例：`0x00578833` 的二进制为：

0000 0000 0101 0111 1000 1000 0011 0011

低 7 位 `0110011` 表示 R-Type 算术指令；`funct7=0000000` 且 `funct3=000` 表示 `add`；字段为 `rs2=x5`、`rs1=x15`、`rd=x16`，因此：

```
add x16, x15, x5
```

## 5.10 立即数编码变体 (Immediate Encoding Variants)

RISC-V 为了让寄存器字段位置尽量固定，把不同格式的立即数拆分到不同指令位中。复习时不要只背“格式图”，还要理解两组关键对比：**S-Type vs. B-Type** 和 **U-Type vs. J-Type**。

### 指令格式中的立即数字段

| 格式 | 31..25 | 24..20 | 19..15 | 14..12 | 11..7 | 6..0 | 用途 |
|---|---|---|---|---|---|---|---|
| R-type | funct7 | rs2 | rs1 | funct3 | rd | opcode | 寄存器算术/逻辑 |
| I-type | imm[11:0] | | rs1 | funct3 | rd | opcode | Load、立即数运算、jalr |
| S-type | imm[11:5] | rs2 | rs1 | funct3 | imm[4:0] | opcode | Store |
| B-type | imm[12,10:5] | rs2 | rs1 | funct3 | imm[4:1,11] | opcode | 条件分支 |
| U-type | imm[31:12] | | | | rd | opcode | lui/auipc |
| J-type | imm[20,10:1,11,19:12] | | | | rd | opcode | jal |

### 从指令位恢复真实立即数

硬件解码时，会从指令位 `inst[31:0]` 中重新拼接立即数：

| 立即数类型 | 拼接方式 |
|---|---|
| I-immediate | {inst[31] 符号扩展, inst[30:20]} |
| S-immediate | {inst[31] 符号扩展, inst[30:25], inst[11:8], inst[7]} |
| B-immediate | {inst[31] 符号扩展, inst[7], inst[30:25], inst[11:8], 0} |
| U-immediate | {inst[31], inst[30:20], inst[19:12], $12'b0$} |
| J-immediate | {inst[31] 符号扩展, inst[19:12], inst[20], inst[30:25], inst[24:21], 0} |

### S-Type vs. B-Type

S-Type 与 B-Type 都使用 12 位立即数字段，并且都保留 `rs1`、`rs2`、`funct3`、`opcode` 的位置。区别在于：

- **S-Type** 用于 Store，立即数表示普通字节偏移：

$$\texttt{imm[11:0]} = \{\texttt{inst[31:25]}, \texttt{inst[11:7]}\}$$

- **B-Type** 用于分支，分支目标按 2 字节粒度对齐，因此最低位恒为 0：

$$\texttt{offset} = \{\texttt{inst[31]}, \texttt{inst[7]}, \texttt{inst[30:25]}, \texttt{inst[11:8]}, 0\}$$

- 这样设计避免在硬件中把整个立即数字段统一左移 1 位；中间位 `imm[10:1]` 和符号位保持在相近位置，而 S-Type 的最低字段位 `inst[7]` 在 B-Type 中变成较高位 `imm[11]`。

### U-Type vs. J-Type

U-Type 与 J-Type 都含 20 位立即数，但用途不同：

- **U-Type** 用于构造高位立即数，真实值为：

$$\texttt{imm[31:12]} << 12$$

  即低 12 位补 0，常见于 `lui` 和 `auipc`。

- **J-Type** 用于 `jal`，真实跳转偏移为：

$$\{\texttt{imm[20]}, \texttt{imm[10:1]}, \texttt{imm[11]}, \texttt{imm[19:12]}, 0\}$$

  即左移 1 位，目标仍按 2 字节边界对齐。

- J-Type 的立即数位顺序看起来“跳跃”，是为了最大化与 U-Type 以及其他格式的重叠，降低解码和布线复杂度。

**易错点：编码值不等于字节偏移**

分支和跳转立即数的最低位隐含为 0，因此机器码中保存的是按 2 字节粒度重排后的偏移信息。做题时要先判断目标指令地址与当前 PC 的差值，再考虑是否需要除以 2 以及如何拆分到 `inst[31]`、`inst[7]`、`inst[30:25]`、`inst[11:8]` 等位置。

# 6 指令编码速查 (Encoding Reference)

## 6.1 常用 R-Type 与 I-Type 编码

| 格式 | 指令 | Opcode | Funct3 | Funct6/7 |
|---|---|---|---|---|
| R | `add` | 0110011 | 000 | 0000000 |
| R | `sub` | 0110011 | 000 | 0100000 |
| R | `sll` | 0110011 | 001 | 0000000 |
| R | `xor` | 0110011 | 100 | 0000000 |
| R | `srl` | 0110011 | 101 | 0000000 |
| R | `sra` | 0110011 | 101 | 0100000 |
| R | `or` | 0110011 | 110 | 0000000 |
| R | `and` | 0110011 | 111 | 0000000 |
| R/A | `lr.d` | 0110011 | 011 | 0001000 |
| R/A | `sc.d` | 0110011 | 011 | 0001100 |
| I | `addi` | 0010011 | 000 | n.a. |
| I | `slli` | 0010011 | 001 | 000000 |
| I | `xori` | 0010011 | 100 | n.a. |
| I | `srli` | 0010011 | 101 | 000000 |
| I | `srai` | 0010011 | 101 | 010000 |
| I | `ori` | 0010011 | 110 | n.a. |
| I | `andi` | 0010011 | 111 | n.a. |
| I | `jalr` | 1100111 | 000 | n.a. |

## 6.2 Load/Store、Branch、U/J-Type 编码

| 格式 | 指令 | Opcode | Funct3 | Funct6/7 |
|---|---|---|---|---|
| I | `lb` | 0000011 | 000 | n.a. |
| I | `lh` | 0000011 | 001 | n.a. |
| I | `lw` | 0000011 | 010 | n.a. |
| I | `ld` | 0000011 | 011 | n.a. |
| I | `lbu` | 0000011 | 100 | n.a. |
| I | `lhu` | 0000011 | 101 | n.a. |
| I | `lwu` | 0000011 | 110 | n.a. |
| S | `sb` | 0100011 | 000 | n.a. |
| S | `sh` | 0100011 | 001 | n.a. |
| S | `sw` | 0100011 | 010 | n.a. |
| S | `sd` | 0100011 | 111 | n.a. |
| B | `beq` | 1100011 | 000 | n.a. |
| B | `bne` | 1100011 | 001 | n.a. |
| B | `blt` | 1100011 | 100 | n.a. |
| B | `bge` | 1100011 | 101 | n.a. |
| B | `bltu` | 1100011 | 110 | n.a. |
| B | `bgeu` | 1100011 | 111 | n.a. |
| U | `lui` | 0110111 | n.a. | n.a. |
| J | `jal` | 1101111 | n.a. | n.a. |

# 7 特权 ISA 与软件栈 (Privileged ISA and Software Stack)

## 7.1 特权模式

RISC-V 特权规范定义了三个主要特权级别，Machine mode 是最高特权级，也是唯一必需模式。

| Level | Encoding | Name | Abbrev. |
|---|---|---|---|
| 0 | 00 | User/Application | U |
| 1 | 01 | Supervisor | S |
| 2 | 10 | Reserved | – |
| 3 | 11 | Machine | M |

高特权模式通常能访问低特权模式的功能，并额外支持中断处理、异常处理和 I/O 等能力。处理器大部分时间运行在最低可用特权级；中断和异常会把控制转交给更高特权级。

## 7.2 实现层次与特权软件栈

不同实现可提供 1 到 4 个特权层次，以隔离能力换取实现成本：

| 层数 | 支持模式 | 用途 |
|---|---|---|
| 1 | M | 简单嵌入式系统 |
| 2 | M, U | 安全嵌入式系统 |
| 3 | M, S, U | 类 Unix 操作系统 |

课件中的软件栈包含 Application、ABI、AEE、OS、SBI、SEE、Hypervisor、HBI、HEE 等层次。需要理解的是：越靠下越接近硬件和特权资源，越靠上越接近应用程序。

常见特权指令包括：

- Machine/Supervisor trap return：从陷入处理返回。
- Supervisor-mode fence virtual memory address：虚拟内存相关 fence。
- Wait for interrupt：等待中断。

# 8 汇编语言与调用约定 (Assembly Language and Calling Convention)

## 8.1 函数调用角色

函数调用中有两个角色：

- **Caller**：调用者，例如 `main` 调用 `sum` 时，`main` 是 caller。它负责传参并跳转到被调函数。
- **Callee**：被调用者，例如 `sum`。它完成任务，把返回值放到约定位置，然后返回调用点。

## 8.2 函数调用六步

RISC-V 函数调用通常可分为六步：

1. 把参数放到 callee 能访问的位置，通常是 `x10`–`x17`。
2. 用 `jal` 跳转到函数，并把返回地址放入 `x1`。
3. 在函数入口分配局部存储，必要时保存寄存器。
4. 执行函数主体。
5. 把返回值放到 caller 能访问的位置，通常是 `x10`–`x17`，恢复寄存器并释放局部存储。
6. 用 `ret` 或 `jalr x0,0(x1)` 返回调用点。

## 8.3 函数入口与出口

- **Prologue**：函数入口代码。若参数和局部变量太多，或需要保护寄存器，就在栈上创建 stack frame。
- **Epilogue**：函数出口代码。撤销栈帧、恢复寄存器并返回。

调用与返回的核心指令：

```
jal  x1, ProcedureLabel     # x1 = PC + 4; PC = target
jalr x0, 0(x1)              # return; x0 discards link
```

## 8.4 寄存器约定

| 名字 | 编号 | 用途 | 调用后保持? |
|---|---|---|---|
| `x0(zero)` | 0 | 常数 0 | n.a. |
| `x1(ra)` | 1 | 返回地址 / link register | 需维护返回路径 |
| `x2(sp)` | 2 | 栈指针 | yes |
| `x3(gp)` | 3 | 全局指针 | yes |
| `x4(tp)` | 4 | 线程指针 | yes |
| `x5-x7(t0-t2)` | 5–7 | 临时寄存器 | no |
| `x8(s0/fp)` | 8 | 保存寄存器 / 帧指针 | yes |
| `x9(s1)` | 9 | 保存寄存器 | yes |
| `x10-x17(a0-a7)` | 10–17 | 参数 / 返回值 | no |
| `x18-x27(s2-s11)` | 18–27 | 保存寄存器 | yes |
| `x28-x31(t3-t6)` | 28–31 | 临时寄存器 | no |

**Saved 与 Temporary 的责任**

`temporary` 寄存器可被 callee 随意使用，返回后 caller 不能假设其值还在。`saved` 寄存器若被 callee 使用，callee 必须保存并恢复。非叶函数若会再次调用其他函数，也必须保护自己的返回地址 `ra`，因为新的 `jal` 会覆盖它。

### 8.5 栈与栈指针

栈适合保存溢出的寄存器、返回地址和局部变量。

- 栈指针 `sp` 为 `x2`。
- RISC-V 栈从高地址向低地址增长。
- Push 8 字节：`sp = sp - 8`。
- Pop 8 字节：`sp = sp + 8`。

### 8.6 叶函数示例

C 代码：

```
long long leaf_example(long long g, long long h,
                       long long i, long long j) {
    long long f;
    f = (g + h) - (i + j);
    return f;
}
```

参数 `g,h,i,j` 在 `x10,x11,x12,x13`，返回值放入 `x10`。课件示例展示了把寄存器压栈、计算并恢复：

```
addi sp, sp, -24
sd   x5, 16(sp)
sd   x6, 8(sp)
sd   x20, 0(sp)
add  x5, x10, x11      # x5 = g + h
add  x6, x12, x13      # x6 = i + j
sub  x20, x5, x6       # f = x5 - x6
addi x10, x20, 0       # return value
ld   x20, 0(sp)
ld   x6, 8(sp)
ld   x5, 16(sp)
addi sp, sp, 24
jalr x0, 0(x1)
```

补充：按常见 ABI，`x5/x6` 是临时寄存器，callee 通常不必为 caller 保存；`x20` 是 saved register，使用后必须恢复。课件示例更强调“保存现场”的栈操作过程。

### 8.7 非叶函数与递归

非叶函数会调用其他函数，因此需要在栈上保存：

- 自己的返回地址 `x1/ra`；
- 调用后还要继续使用的参数和临时值；
- 被使用的 saved registers。

递归阶乘例子：

```
long long fact(long long n) {
    if (n < 1) return 1;
    else return n * fact(n - 1);
}
```

核心汇编结构：

```
fact:
    addi sp, sp, -16
    sd   x1, 8(sp)
    sd   x10, 0(sp)
    addi x5, x10, -1
    bge  x5, x0, L1
    addi x10, x0, 1
    addi sp, sp, 16
    jalr x0, 0(x1)
L1:
    addi x10, x10, -1
    jal  x1, fact
    add  x6, x10, x0
    ld   x10, 0(sp)
    ld   x1, 8(sp)
    addi sp, sp, 16
    mul  x10, x10, x6
    jalr x0, 0(x1)
```

- 保存 `x10`：递归调用会把返回值也放到 `x10`，原来的 $n$ 会丢失。
- 保存 `x1`：递归调用中的 `jal x1,fact` 会覆盖当前函数返回到 caller 的地址。
- 基例 `n<1` 返回 1；递归例先调用 `fact(n-1)`，再乘以保存的 $n$。

### 8.8 递归的缺点

- 每层递归都要保存处理器状态，可能造成栈溢出 (stack overflow)。
- Push/Pop 消耗时间和内存空间。
- 可用循环或尾调用优化减少递归开销。

### 8.9 调用前后保持与不保持的内容

- **保持**：saved registers、`sp`、返回路径所需的 `ra`、栈指针以上的栈内容。
- **不保持**：temporary registers、参数寄存器 `x10`–`x17`、返回值寄存器 `x10`–`x17`、栈指针以下的区域。

### 8.10 C 变量存储类别与内存布局

C 变量常见存储类别：

- **automatic**：自动变量，通常在栈帧中。
- **static**：静态变量，通常在静态数据区。

内存布局自低地址到高地址通常为：

| 区域 | 内容 |
|---|---|
| Reserved | 低地址保留区。 |
| Text | 程序代码；PC 指向此区域中的指令。 |
| Static data | 全局变量、`static` 变量、常量数组和字符串；`gp(x3)` 初始化为可用正负偏移访问该区域的地址。 |
| Dynamic data / heap | 动态分配区，如 C 的 `malloc`、Java 的 `new`。 |
| Stack | 自动存储区，从高地址向低地址增长；`sp` 指向当前栈顶。 |
| OS | 用户地址空间顶部附近可能由操作系统保留。 |

### 8.11 栈帧 (Stack Frame)

栈帧是函数入口分配、函数退出释放的私有空间，也称 activation record。课件栈帧包含：

- 返回地址 (Return Address)。
- 旧帧指针 (Old FP)。
- 保存寄存器 (Saved Registers)。

- 局部变量 (Local Variables)。
- 传给被调函数的额外参数。

`fp(x8)` 可作为稳定基准访问栈帧内容，`sp` 随压栈/出栈变化。栈溢出与安全攻击密切相关，课件提到经典文章 “Smashing The Stack For Fun And Profit”。

# 9 从源代码到运行程序 (From Source Code to A Running Program)

## 9.1 整体流程

C 程序变为正在运行的进程，逻辑上经过以下步骤：

$$\texttt{foo.c} \xrightarrow{\text{Compiler}} \texttt{foo.s} \xrightarrow{\text{Assembler}} \texttt{foo.o} \xrightarrow{\text{Linker + libraries}} \texttt{a.out} \xrightarrow{\text{Loader}} \text{running process}$$

实际系统中某些步骤可能被合并以加快翻译，但理解这条链路有助于定位编译、链接和运行时错误。

## 9.2 编译器与预处理器

- **Preprocessor** `*.c` → `*.i`：展开宏定义、处理 `#include` 和其他以 `#` 开头的指令。
- **Compiler** `*.i` → `*.s`：把预处理后的 C 程序转换为汇编语言。

课件中说明编译器内部细节不在本课展开。

## 9.3 汇编器与伪指令

汇编器不仅把处理器能直接执行的指令翻译成机器码，还支持对程序员和编译器更方便的伪指令 (pseudo-instructions)。伪指令会被展开为一条或多条基础指令。

| 伪指令 | 可能展开 | 含义 |
|---|---|---|
| `lla rd,symbol` | `auipc rd,symbol[31:12]` + `addi rd,rd,symbol[11:0]` | 加载本地地址 |
| `la rd,symbol` | PIC: 经 GOT 加载；Non-PIC: 类似 `lla` | 加载地址 |
| `li rd,imm` | 若立即数较大，展开为多条指令 | 加载立即数 |
| `mv rd,rs` | `addi rd,rs,0` | 复制寄存器 |
| `not rd,rs` | `xori rd,rs,-1` | 按位取反 |
| `seqz rd,rs` | `sltiu rd,rs,1` | 等于 0 则置 1 |
| `bgt rs,rt,L` | `blt rt,rs,L` | 大于则分支 |
| `ble rs,rt,L` | `bge rt,rs,L` | 小于等于则分支 |
| `j L` | `jal x0,L` | 无条件跳转 |
| `ret` | `jalr x0,0(x1)` | 函数返回 |
| `call offset` | `auipc x1,offset[31:12]` + `jalr x1,x1,offset[11:0]` | 远距离函数调用 |

### 9.4 汇编指示符 (Assembler Directives)

汇编指示符指导汇编器组织目标文件，但本身不是处理器执行的指令。

- `.text`：进入代码段。
- `.align 2`：把后续代码对齐到 $2^2$ 字节。
- `.globl main`：声明全局符号 `main`。
- `.section .rodata`：进入只读数据段。
- `.balign 4`：把数据段对齐到 4 字节。
- `.string "Hello, %s!\n"`：创建以 `NUL` 结尾的字符串。

### 9.5 Hello World 汇编结构

C 代码：

```
#include <stdio.h>
int main() {
    printf("Hello, %s!\n", "world");
    return 0;
}
```

对应汇编框架包含：

```
.text
.align 2
.globl main
main:
    addi sp, sp, -16
    sw   ra, 12(sp)
    lui  a0, %hi(string1)
    addi a0, a0, %lo(string1)
    lui  a1, %hi(string2)
    addi a1, a1, %lo(string2)
    call printf
    lw   ra, 12(sp)
    addi sp, sp, 16
    li   a0, 0
    ret
```

```
.section .rodata
.balign 4
string1:
    .string "Hello, %s!\n"
string2:
    .string "world"
```

汇编器输出 `*.o` 时采用 ELF 格式。目标文件中会出现机器码、符号表、重定位信息等。

# 10 ELF、链接与装载 (ELF, Linking and Loading)

## 10.1 ELF 基础

ELF (Executable and Linkable Format) 是 Unix 和类 Unix 系统中常见的可执行文件、目标文件、共享库与 core dump 的标准格式。它灵活、可扩展、跨平台。

ELF 文件有两种视图：

- **Section view**：面向链接，section header 列出各 section，例如 `.text`、`.data`、`.rodata`、符号表、重定位信息等。
- **Segment view**：面向运行，program header 描述装载时使用的 segment。

## 10.2 三类目标文件

1. **Relocatable file**：包含适合与其他目标文件链接的代码和数据，例如 Linux 的 `.o`。
2. **Executable file**：可执行程序，描述操作系统如何创建进程映像。
3. **Shared object file**：共享对象，可参与静态链接生成其他目标文件，也可在运行时由动态链接器装入进程。

## 10.3 可重定位目标文件

汇编器输出的可重定位目标文件具有以下特点：

- 是二进制机器码，但还不能直接执行。
- 可能引用外部符号，因此需要符号表。
- 每个目标文件有自己的地址空间，地址之后需要由链接器修正。

典型内容包括：

- Header：文件各部分大小与位置。

- Text Section：指令。
- Data Section：静态数据，如局部/全局变量、字符串、常量。
- Debugging Information：源代码行号到代码地址映射等。
- Symbol Table：导出引用和未解析的导入引用。
- Relocation Information：哪些指令或数据依赖最终地址，需要之后修补。

### 10.4 链接器的作用

链接器把多个 `.o` 和库例程合成为一个可执行文件。这样一个文件改动时，只需重新编译/汇编改动模块，再与已有机器码模块重新链接。

链接要解决三个核心问题：

1. **合并目标文件**：把各目标文件的 text、data、rodata 等同类段合并。
2. **符号解析**：把未定义的外部引用绑定到某个模块中定义的符号。
3. **地址重定位**：修正代码和数据中依赖最终地址的位置。

### 10.5 地址修正与符号解析

目标文件中的地址通常相对于本文件代码段或数据段起点。当多个目标文件合并后，地址必须重新计算。

- **Fixing Addresses**：链接器根据重定位信息修补地址字段。
- **Symbol Resolution**：若模块 A 引用模块 B 中定义的 `x`，链接器必须确定合并后 `x` 的位置，并修改所有对 `x` 的引用。

每个可链接模块的符号表通常包含：

- 本模块定义的全局符号；
- 本模块引用但未定义的全局符号，即 externals；
- 段名，如 text、data、rodata；
- 可选的非全局符号和调试信息。

### 10.6 可执行文件格式

可执行目标文件包含：

- Header：记录 `main` 或更底层入口点位置。
- Text Segment：指令。
- Data Segment：静态数据。
- Relocation Information：若仍保留重定位需求，则指出依赖实际地址的指令和数据。
- Symbol Table：导出和导入引用。
- Debugging Information。

### 10.7 静态链接与动态链接

| 类型 | 优点 | 缺点 |
|---|---|---|
| 静态链接 | 程序启动简单；库代码在链接时确定；运行时依赖少。 | 若多个程序都链接同一库，会浪费磁盘/内存；库更新后已静态链接的程序仍使用旧版本。 |
| 动态链接 | 外部函数首次调用时才装入/链接；多个程序可共享同一库；每次启动可使用当前库版本。 | 首次调用有额外开销；运行时需要查符号表并解析；实现更复杂。 |

### 10.8 静态库与共享库

- **静态库**：对象文件集合，可类比压缩包。例如 `libc.a` 内含 `printf.o`、`read.o`、`exit.o` 等。链接器只选择解析未定义引用所需的对象文件。
- **共享库**：多个可执行文件指向磁盘上的同一个共享库，最终链接和重定位由 loader 完成。优化方法包括固定非零装载地址、跳转表、运行时 patch jump 等。

### 10.9 装载器 (Loader)

`a.out` 是保存在存储设备中的可执行文件；运行程序时，loader 的工作是把它装入内存并跳到起始地址。现代系统中，loader 通常是操作系统的一部分。

Loader 负责：

- 初始化寄存器、栈、程序参数和环境变量。
- 根据头部确定所需地址空间大小。
- 分配地址空间并读入代码/数据 segment。
- 把未初始化数据 `.bss` 清零。

- 创建栈段。
- 设置运行时信息并从入口点开始执行。

### 10.10 动态装载、PIC 与 GOT

动态链接程序的装载更复杂：操作系统先启动动态链接器，动态链接器再启动目标程序，并在第一次外部调用时装入和修补目标函数。课件特别提到 **GOT** 与 **PLT**：

- **PIC (Position-Independent Code)**：代码不依赖固定装载地址，适合共享库。
- **PC-relative addressing**：通过 PC 相对寻址获取当前位置，是生成 PIC 的基础。
- **GOT (Global Offset Table)**：链接器创建的全局偏移表，保存全局数据或函数地址入口。
- **PLT / stub**：程序先跳到短 stub；首次调用时 stub 进入动态链接器解析符号，之后可改写入口以快速跳到真实函数。

ELF 中数据页跟在代码页后，代码到数据的偏移不依赖实际装载地址。若程序能把自己的地址装入寄存器，就能用固定偏移访问 GOT，再由 GOT 找到数据。

### 10.11 Lazy Linkage

Lazy linkage 延迟符号解析：

1. 第一次调用动态库例程时，stub 装入例程 ID 并跳到 linker/loader。
2. 动态链接器检查目标例程是否已装入。
3. 若未装入，则装入并链接；若已装入，则直接绑定当前引用。
4. 后续调用通过已修补的间接表或跳转入口快速到达目标例程。

## 11 补充主题与课件结尾 (Additional Topics)

### 11.1 编程语言演进与自举

课件用编程语言演进图引出“编译器如何产生自己”的问题。自举 (bootstrapping) 指用某种语言编写该语言自己的编译器：

- 初始核心编译器可用另一种语言或汇编写成。
- 之后逐步扩展语言子集，并用已有编译器编译新版本。
- 这解决了编译器设计中的“先有鸡还是先有蛋”问题。

### 11.2 Reflections on Trusting Trust

课件提到 Ken Thompson 的 “Reflections on Trusting Trust”。核心思想是：即使源代码看起来干净，编译器本身也可能在编译时植入后门，并在编译新编译器时继续复制这种行为。因此，系统可信性不仅取决于源代码，还取决于工具链、构建过程和二进制来源。

### 11.3 真实入口点：_start 与 crt0

对于大多数 C/C++ 程序，真正入口点不是 `main`，而是 `_start`。它由 C runtime 启动代码实现，传统上称为 `crt0` 或 `c0`。

`crt0` 的任务包括：

- 初始化全局指针 `gp`。
- 清空 `.bss` 段。
- 注册退出时执行的终止函数。
- 运行全局初始化函数。
- 设置 `argc`、`argv`、`envp`。
- 调用 `main`，然后调用 `exit`。

课件中的 `crt0.s` 例子展示了 `.text`、`.global _start`、`la gp,__global_pointer$`、清 `bss`、调用 `atexit`、`call main` 与 `tail exit` 等启动流程。

### 11.4 Quiz：符号放在哪里?

题目代码包含：

```
#define ITEM_NUM 16
static int ITEM_SIZE = 4;

int main() {
    size_t buf_size = ITEM_NUM * ITEM_SIZE * sizeof(char);
    char* heap_buf = (char *) malloc(buf_size);
    if (heap_buf) {
        printf("Succeed to allocate: %d!\n", buf_size);
    }
    free(heap_buf);
    return 0;
}
```

选项：A. Text Section；B. Data Section；C. Exported reference in symbol table；D. Imported reference in symbol table； E. None of the above。

| 问题 | 答案 | 原因 |
|---|---|---|
| `ITEM_NUM` | E | 宏在预处理阶段被替换为常量 16，不作为目标文件符号存在。 |
| `ITEM_SIZE` | B | `static int` 是具有静态存储期的已初始化数据，放入数据段；其符号通常是本地符号，不是导出引用。 |
| `malloc` | D | 当前目标文件引用但不定义 `malloc`，需要由链接器从库中解析，因此是导入引用。 |

## 12 全章总结 (Summary)

本讲从 RISC-V ISA 进入汇编语言与程序装载链路，复习时可以抓住以下主线：

1. **RISC-V 是开放、简单、可扩展的 ISA**：基础整数集小，通用能力通过 `M/A/F/D/C/G` 等扩展补足。
2. **编码格式是考试核心**：R/I/S/B/U/J 六类格式、寄存器字段固定、立即数符号位在第 31 位、B/J 偏移最低位隐含 0。
3. **Load/Store 是内存访问唯一通路**：ALU 只操作寄存器，Load/Store 使用基址加位移；注意符号扩展、零扩展、字节序和对齐。
4. **控制转移依赖 PC 相对寻址**：掌握 `jal/jalr`、条件分支、far branch 改写、while/if/switch 的汇编模式。
5. **调用约定规定谁保存什么**：`a` 寄存器传参和返回，`t` 寄存器不保持，`s` 寄存器由 callee 保持，非叶函数要保护 `ra`。
6. **栈帧连接高级语言和机器执行**：局部变量、保存寄存器、返回地址和参数溢出都通过栈帧管理。
7. **从源代码到运行程序是一条工具链**：预处理、编译、汇编、链接、装载分别解决不同问题；ELF、符号表、重定位、GOT/PLT 和 lazy linkage 是理解链接错误与运行时绑定的关键。
8. **真实入口点早于 `main`**：`_start/crt0` 完成运行时初始化，再调用 `main` 并进入 `exit`。

“*ISA 是软件看见的硬件；汇编、链接和装载则把高级语言程序一步步落到这层硬件抽象上。*”